

Manual Definitivo: FSelenium

Índice General

Bienvenido al manual completo de automatización. Haz clic en cualquier sección para saltar directamente a ella.

Fundamentos de Java y POO para QA Automation

- [1. Tipos de Datos y Estructuras de Control](#)
- [2. Programación Orientada a Objetos \(POO\)](#)
- [3. Colecciones de Datos \(List vs Set\)](#)
- [4. Excepciones: try-catch-finally](#)
- [5. Variables Estáticas vs De Instancia](#)
- [6. Buenas Prácticas y Métodos Eficientes](#)
- [7. Ejercicios Prácticos](#)

Curso Intensivo Paso a Paso: Selenium, Java y Cucumber BDD

- [Módulo 1: Fundamentos y Configuración Inicial](#)
- [Módulo 2: Cucumber y BDD](#)
- [Módulo 3: Profundización en POM](#)
- [Módulo 4: TestNG y Aserciones](#)
- [Módulo 5: Interacciones Complejas](#)
- [Módulo 6: Estándares Arquitectónicos](#)

Manual de Estrategias de Localizadores en Selenium

- [1. Introducción a los Localizadores](#)
- [2. Estrategias Básicas de Localización](#)
- [3. Selectores Avanzados: CSS Selectors](#)
- [4. Selectores Avanzados: XPath](#)
- [5. CSS Selector vs XPath](#)
- [6. Mejores Prácticas de QA](#)

Explicaciones, Tips y Buenas Prácticas

- [1. El Flujo de Ejecución de Cucumber \(BDD\)](#)
 - [2. El Patrón Page Object Model \(POM\) con Herencia](#)
 - [3. Manejo de Esperas: La Regla de Oro](#)
 - [4. Consejos de Depuración en IntelliJ](#)
-

Fundamentos de Java y POO para QA Automation

Este manual está diseñado para llevarte paso a paso por los conceptos fundamentales de Java y la Programación Orientada a Objetos (POO), destilando los conceptos clave orientados exclusivamente a resolver los problemas del día a día en la automatización de pruebas con Selenium WebDriver.

1. Tipos de Datos y Estructuras de Control

Antes de interactuar con el navegador, es crucial dominar el lenguaje.

1.1 Tipos de Datos y Variables

En Java, cada variable debe tener un tipo declarado.

- **Primitivos:** `int` (enteros), `double` (decimales), `boolean` (verdadero/falso), `char` (carácter).
- **De Referencia:** `String` (texto), `List`, objetos de clases personalizadas.

```
String url = "https://www.ejemplo.com";
int timeoutMilisegundos = 5000;
boolean esVisible = true;
```

1.2 Estructuras de Control y Bucles

Iterar es el pan de cada día en QA, especialmente al interactuar con Tablas o Grillas web.

- **if / else:** Para tomar decisiones basadas en condiciones.
- **Bucle for Clásico:** Útil cuando necesitas interactuar con elementos mediante su índice.
- **Bucle for-each (Recomendado):** Más limpio cuando solo quieres iterar sobre los elementos.

```
// Ejemplo iterando sobre una lista de WebElements
List<WebElement> botones = driver.findElements(By.className("btn"));
for(WebElement boton : botones) {
    if(boton.getText().equals("Aceptar")) {
        boton.click();
        break; // Detiene el bucle inmediatamente
    }
}
```

[🔙 Volver al Índice General](#)

2. Programación Orientada a Objetos (POO)

La POO nos permite modelar las páginas web y nuestros tests como "objetos", haciendo el código más mantenible.

2.1 Clases vs Objetos

- **Clase:** Es un "plano" o plantilla. Por ejemplo, la clase `PaginaLogin`.

- **Objeto (Instancia):** Es la "casa" construida a partir de ese plano. Cuando en tu test escribes `new PaginaLogin()` , estás instanciando un objeto real.

2.2 Encapsulamiento y Modificadores de Acceso

Ocultar los detalles internos y mostrar solo lo necesario.

- **public** : Cualquier clase del proyecto puede verlo. Los métodos (acciones) como `iniciarSesion()` siempre deben ser **public** .
- **private** : Solo la propia clase puede verlo. **Tus localizadores (XPath, ID) SIEMPRE deben ser private** .
- **protected** : Solo la clase y las que "heredan" de ella pueden verlo.

2.3 Herencia y la palabra **super**

La herencia (**extends**) permite que una clase herede métodos de una clase "padre". Es la base del **Page Object Model**. En Selenium, creamos una **BasePage** que contenga métodos comunes (esperas, inicialización del driver) para que las demás páginas hereden de ella.

```
// CLASE PADRE
public class BasePage {
    protected WebDriver driver;

    public BasePage(WebDriver driver) {
        this.driver = driver;
    }
}

// CLASE HIJA
public class HomePage extends BasePage {
    public HomePage(WebDriver driver) {
        super(driver); // Invoca al constructor del padre
    }
}
```

2.4 Polimorfismo

Capacidad de usar una misma interfaz para diferentes tipos subyacentes.

- **En Selenium:** **WebDriver** es una interfaz. Podemos inicializarla como **ChromeDriver** , **FirefoxDriver** , y el resto del código no cambia.

[📖 Volver al Índice General](#)

3. Colecciones de Datos (**List** vs **Set**)

Cuando le pides a Selenium múltiples elementos (usando `findElements`), te devuelve una Colección de tipo **List** .

- **List (Listas):** Ordenadas y permiten duplicados. Cada elemento tiene un índice (0, 1, 2...).
 - *Uso QA:* `List<WebElement>` para extraer resultados de una búsqueda.
- **Set (Conjuntos):** Desordenados y **no permiten duplicados**.

- *Uso QA:* Crítico para manejar Múltiples Pestañas. `driver.getWindowHandles()` devuelve un `Set<String>` con los identificadores únicos de cada ventana abierta.

[↑ Volver al Índice General](#)

4. Excepciones: `try-catch-finally`

En Selenium, los elementos a veces no cargan, las conexiones fallan, o el diseño de la página cambia. Si no capturas una excepción, tu suite de pruebas se detiene.

```
public void clicSeguro(By locator) {  
    try {  
        // INTENTA hacer esto  
        driver.findElement(locator).click();  
  
    } catch (ElementClickInterceptedException e) {  
        // SI FALLA por estar interceptado, haz esto (Plan B)  
        JavascriptExecutor js = (JavascriptExecutor) driver;  
        js.executeScript("arguments[0].click();", driver.findElement(locator));  
  
    } finally {  
        // Esto se ejecuta SIEMPRE, haya fallado o no.  
        System.out.println("Terminando intento de clic.");  
    }  
}
```

[↑ Volver al Índice General](#)

5. Variables Estáticas vs De Instancia

Una variable normal le pertenece **exclusivamente al objeto** que fue creado. Una variable `static` le pertenece **a la Clase** y es compartida universalmente.

Por qué NO usar `public static WebDriver driver;`

Si declaras tu navegador como estático y corres pruebas en paralelo, el hilo 1 y el hilo 2 chocarán compartiendo el mismo navegador. Todo colapsa. **La Solución Senior:** Usar `ThreadLocal<WebDriver>`. Esto permite que cada hilo de ejecución tenga una copia secreta e independiente del driver.

[↑ Volver al Índice General](#)

6. Buenas Prácticas (SOLID, DRY, KISS) y Métodos Eficientes

6.1 DRY (No te repitas) y KISS (Mantenlo simple)

Si copias y pegas código, algo está mal. Extrae la lógica repetida a métodos o clases base. Nombra variables de forma descriptiva.

6.2 Principios SOLID aplicados a QA

- **Single Responsibility:** Un Page Object solo debe representar una página web. No debe contener lógica de testeo ni aserciones (Asserts). Las aserciones van en la clase Test.
- **Open/Closed:** El código debe estar abierto a extensión pero cerrado a modificación.

6.3 Métodos "Wrappers" (Envoltorios)

En lugar de usar los métodos nativos de Selenium en crudo, "envuélvelos" en métodos propios en tu `BasePage` para agregar esperas inteligentes.

```
public void escribirTexto(By locator, String texto) {  
    WebElement elemento = wait.until(ExpectedConditions.visibilityOfElementLocated(locator));  
    elemento.clear();  
    elemento.sendKeys(texto);  
}
```

[↑ Volver al Índice General](#)

7. Ejercicios Prácticos

Ejercicio 1: Clases y Encapsulamiento

Objetivo: Crear una clase `Usuario` para representar datos de prueba, con propiedades privadas `nombreUsuario` y `password`, un constructor y métodos Getters.

Ejercicio 2: Refactorización

Objetivo: Crear un método en una clase `BasePage` llamado `public void clicar(By localizador)` que incluya una espera explícita `WebDriverWait` antes de hacer el clic real.

Ejercicio 3: Implementando POM Completo

Objetivo: Crear una clase `GoogleSearchPage` que herede de `BasePage`, definiendo los localizadores de forma privada y exponiendo un método público `buscarTexto(String texto)`.

Dominar Java es el verdadero salto de calidad entre grabar pruebas (Record & Playback) y ser un Ingeniero de Automatización.

[↑ Volver al Índice General](#)

Bienvenido al curso definitivo de automatización de pruebas integradas. Este manual está diseñado como un libro de texto guiado. Al finalizar, serás capaz de construir arquitecturas de QA sólidas y escalables.

 **Pre-requisito:** Antes de dominar la arquitectura, debes saber cómo interactuar con el HTML. Por favor, lee de forma obligatoria el Manual de Localizadores adjunto en este framework.

Módulo 1: Fundamentos y Configuración Inicial (Paso a Paso)

1.1 Entendiendo Selenium WebDriver

Selenium no "testea" nativamente. Selenium simplemente provee una API que se comunica directamente con el navegador utilizando el **Protocolo W3C WebDriver**. Cuando en Java escribimos `.click()`, Selenium traduce eso en un comando HTTP estandarizado que el navegador entiende y ejecuta.

1.2 Paso 1: Configurar el proyecto (Maven)

Maven es el corazón de la gestión en Java. En lugar de descargar librerías manualmente, usamos el archivo `pom.xml`.

¿Cómo debe lucir un pom.xml básico?

```
<dependencies>
  <!-- El núcleo de Selenium -->
  <dependency>
    <groupId>org.seleniumhq.selenium</groupId>
    <artifactId>selenium-java</artifactId>
    <version>4.23.1</version>
  </dependency>
  <!-- Motor de Pruebas -->
  <dependency>
    <groupId>org.testng</groupId>
    <artifactId>testng</artifactId>
    <version>7.11.0</version>
    <scope>test</scope>
  </dependency>
  <!-- Gestor Automático de Navegadores -->
  <dependency>
    <groupId>io.github.bonigarcia</groupId>
    <artifactId>webdrivermanager</artifactId>
    <version>6.3.3</version>
  </dependency>
</dependencies>
```

1.3 Paso 2: Hola Mundo en Selenium (La forma rústica)

Antes de construir arquitecturas complejas, veamos cómo se abre una página de forma directa:

```
import io.github.bonigarcia.wdm.WebDriverManager;
import org.openqa.selenium.WebDriver;
import org.openqa.selenium.chrome.ChromeDriver;

public class HolaMundoSelenium {
```

```

public static void main(String[] args) {
    // 1. Descarga el driver adecuado para tu versión de Chrome
    WebDriverManager.chromedriver().setup();

    // 2. Levanta el navegador (Sesión)
    WebDriver driver = new ChromeDriver();

    // 3. Maximiza la ventana y navega
    driver.manage().window().maximize();
    driver.get("https://www.google.com");

    // 4. Imprime el título para verificar
    System.out.println("El título es: " + driver.getTitle());

    // 5. Destruye la sesión (Crítico para no consumir RAM infinita)
    driver.quit();
}
}

```

[🔙 Volver al Índice General](#)

Módulo 2: Cucumber y el Diseño Basado en Comportamientos (BDD)

BDD acorta la brecha entre desarrolladores, QA y negocio (Product Owners). Las pruebas se escriben en lenguaje humano (Gherkin).

2.1 Escribiendo Escenarios (Archivos `.feature`)

```

Feature: Autenticación de Usuario
  Como cliente registrado
  Quiero iniciar sesión
  Para acceder a mi panel privado

  Scenario: Iniciar sesión exitosamente
    Given que navego a la página de login
    When ingreso el usuario "admin@test.com" y la clave "12345"
    Then el panel de control es visible

```

2.2 Paso a Paso: Mapeando Gherkin a Java (Step Definitions)

Cada oración del archivo `.feature` debe coincidir con un método en Java.

```

public class LoginSteps {

    // El texto dentro de @Given debe coincidir EXACTAMENTE con el .feature
    @Given("que navego a la página de login")
    public void navegarAlLogin() {
        // Aquí interactuaremos con el navegador más adelante
        System.out.println("Navegando...");
    }
}

```

```
// Pasando parámetros desde Gherkin (usando {string})
@When("ingreso el usuario {string} y la clave {string}")
public void ingresarCredenciales(String user, String password) {
    System.out.println("Login con: " + user + " y " + password);
}
}
```

[↑ Volver al Índice General](#)

Módulo 3: Profundización Absoluta en Page Object Model (POM)

3.1 El problema del Código Espagueti (Lo que NO debes hacer)

Imagina tener 50 pruebas que inician sesión así:

```
// MAL EJEMPLO
driver.findElement(By.id("user_email")).sendKeys("test@test.com");
driver.findElement(By.id("user_password")).sendKeys("123");
driver.findElement(By.cssSelector("button[type='submit']")).click();
```

Si el desarrollador cambia el ID `user_email` a `email_address`, ¡tendrás que corregir 50 archivos!

3.2 Paso a Paso: Refactorizando al Patrón POM

El POM centraliza la página en un solo archivo.

Paso 1: Crear la clase de la página (La Vista)

```
public class PaginaLogin {
    // 1. Encapsulamiento: Los localizadores son privados.
    private By emailInput = By.id("user_email");
    private By passwordInput = By.id("user_password");
    private By submitButton = By.cssSelector("button[type='submit']");

    // 2. WebDriver inyectado
    private WebDriver driver;

    public PaginaLogin(WebDriver driver) {
        this.driver = driver;
    }

    // 3. Acciones de negocio (Públicas)
    public void ingresarCredenciales(String email, String pass) {
        driver.findElement(emailInput).sendKeys(email);
        driver.findElement(passwordInput).sendKeys(pass);
        driver.findElement(submitButton).click();
    }
}
```


Paso 2: Herencia (El nivel Senior) En nuestro framework **FSelenium**, no queremos inyectar el **WebDriver** y llamar a **driver.findElement** mil veces. Creamos una **BasePage** que hace el trabajo sucio.

```
// Así luce en FSelenium (Elegante y Limpio)
public class PaginaLogin extends BasePage {
    private By emailInput = By.id("user_email");

    public void ingresarEmail(String email) {
        // write() es heredado de BasePage, ya incluye esperas explícitas automáticas.
        write(emailInput, email);
    }
}
```

[↑ Volver al Índice General](#)

Módulo 4: TestNG y Aserciones Avanzadas

4.1 Aserciones (Validando resultados)

Una prueba no es una prueba si no verifica algo. TestNG nos da aserciones poderosas.

```
// Hard Assert: Si esto falla, la prueba se aborta INMEDIATAMENTE.
Assert.assertEquals(textoObtenido, "Bienvenido", "El texto no coincide.");
Assert.assertTrue(elemento.isDisplayed(), "El botón debería estar visible.");

// Soft Assert: Si falla, la prueba CONTINÚA y reporta todas las fallas al final.
SoftAssert softAssert = new SoftAssert();
softAssert.assertEquals(color, "Rojo");
softAssert.assertTrue(estaVisible);
softAssert.assertAll(); // Crítico: Si no llamas esto, la prueba siempre pasa.
```

4.2 Ejecución Parametrizada (@DataProvider)

Permite correr un test de TestNG múltiples veces con datos distintos.

```
@DataProvider(name = "credenciales")
public Object[][] datosLogin() {
    return new Object[][] {
        {"admin@test.com", "1234"},
        {"user@test.com", "pass123"},
        {"bloqueado@test.com", "0000"}
    };
}

// El test se ejecutará 3 veces automáticamente
@Test(dataProvider = "credenciales")
public void testLogin(String user, String pass) {
    PaginaLogin login = new PaginaLogin();
```

```
login.iniciarSesion(user, pass);
}
```

[↑ Volver al Índice General](#)

Módulo 5: Interacciones Complejas en Selenium

5.1 Formularios: La Clase `Select`

Para interactuar con listas desplegables (Dropdowns) nativas `<select>`.

```
WebElement dropdown = driver.findElement(By.id("paises"));
Select selectPais = new Select(dropdown);

// 3 formas de seleccionar:
selectPais.selectByVisibleText("Argentina"); // Lo que ve el usuario (Mejor)
selectPais.selectByValue("ARG");             // Por el atributo 'value' HTML
selectPais.selectByIndex(2);                  // La tercera opción (Poco seguro)
```

5.2 Estrategias de Espera (Sincronización)

 **ERROR COMÚN:** Usar `Thread.sleep(5000)`. Congela todo y hace la ejecución muy lenta.

Opción A: Implicit Wait (Global)

```
// Espera HASTA 10 segundos intentando buscar elementos.
// Si aparece en el segundo 2, avanza inmediatamente.
driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
```

Opción B: Explicit Wait (Específico) La mejor práctica absoluta. Espera por una condición.

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
// Esperar hasta que un botón invisible se vuelva clicable
WebElement btn = wait.until(ExpectedConditions.elementToBeClickable(By.id("submit")));
btn.click();
```

5.3 Simulación de Mouse: Clase `Actions`

Para arrastrar y soltar (Drag and Drop), mover el mouse por encima (Hover), etc.

```
Actions actions = new Actions(driver);
WebElement arrastrable = driver.findElement(By.id("caja1"));
WebElement destino = driver.findElement(By.id("caja2"));

// Arrastrar y Soltar
```

```
actions.dragAndDrop(arrastrable, destino).perform();

// Mover el mouse por encima de un menú para que se despliegue
WebElement menu = driver.findElement(By.id("menu-desplegable"));
actions.moveToElement(menu).perform();
```

[🔙 Volver al Índice General](#)

Módulo 6: Estándares Arquitectónicos de un QA Senior

- Gestión de Hilos (Thread-Safety):** Si corres pruebas en paralelo, el hilo 1 y el hilo 2 chocarán si comparten una variable `static WebDriver driver`. Usamos `ThreadLocal<WebDriver>` en nuestra clase `DriverManager` para que cada hilo tenga su propia copia del navegador, aislada de las demás.
- Abstracción Total de URLs:** No debe haber ni un solo `http://...` en tu código Java. Se lee todo dinámicamente usando una clase custom como `ConfigReader.java` que extrae los datos de `config.properties`.
- Manejo de Errores Silencioso:** En los *Hooks* de Cucumber, si un escenario falla (`scenario.isFailed()`), inyectamos un bloque `try-catch` que toma un *Screenshot* nativo en bytes y lo pega en el reporte final automáticamente, sin detener abruptamente el flujo de limpieza.

[🔙 Volver al Índice General](#)

Apéndices del Framework (Crecimiento Futuro)

Apéndice A: Data-Driven Avanzado

Para escenarios donde el `@DataProvider` no alcanza, el framework incluye la carpeta `dataloader/`. Usando la librería **Apache POI** para Excel o **OpenCSV** para archivos de texto, puedes cargar miles de filas para pruebas transaccionales masivas, inyectando esos valores a tus variables durante la inicialización de la prueba.

Apéndice B: CI/CD y Headless Testing

Cuando se integran las pruebas en GitHub Actions o Jenkins, la máquina virtual no tiene pantalla. El navegador debe iniciar en "Headless Mode" (invisible).

```
ChromeOptions options = new ChromeOptions();
options.addArguments("--headless", "--window-size=1920,1080"); // Window size crítico para
responsividad
driver = new ChromeDriver(options);
```

Apéndice C: Estrategias de Debugging en IDE

Cuando recibes una excepción inentendible como `StaleElementReferenceException`:

- El DOM se recargó y el `WebElement` que guardaste en una variable quedó "viejo".
- Solución:** Pon un *Breakpoint* (Punto de interrupción rojo) en IntelliJ, ejecuta en modo *Debug*, y usa *Evaluate Expression* para forzar una nueva búsqueda de ese elemento en vivo.

Apéndice D: Integración de API (RestAssured)

Un framework Senior no automatiza precondiciones a través de la Interfaz Gráfica de Usuario (GUI) si es lento.

- **Caso:** Necesito probar cómo borrar un post.
- **Malo:** Selenium navega a "Crear post", llena formularios, y luego va a borrarlo. (Tarda 1 minuto).
- **Senior:** Usa RestAssured (HTTP Client Java) en el `@Before` para enviar un JSON por POST, creando el artículo en la Base de Datos en milisegundos. Luego, abre Selenium directo en la URL del post y hace clic en Borrar.

Apéndice E: Escalabilidad con Selenium Grid

Cuando 100 pruebas demoran demasiado en correr en una PC, usamos Grid. Levantas un Hub central y Nodos satélites (usualmente con contenedores Docker). En el código, sustituyes `new ChromeDriver()` por `new RemoteWebDriver(new URL("http://hub:4444"), options)`. Esto deriva las pruebas a las nubes.

Apéndice F: El Patrón Factory

Para escalar el soporte de navegadores (Chrome, Edge, Firefox, Safari, Appium Mobile), en lugar de tener un `switch-case` interminable, se aplica el Patrón de Diseño Factory. Se crea una interfaz `BrowserFactory` y múltiples implementaciones (ej: `ChromeManager`, `FirefoxManager`). El framework decide en tiempo de ejecución qué manejador instanciar basándose en las variables de entorno.

[↑ Volver al Índice General](#)

Aprender a localizar elementos de manera precisa en el DOM (Document Object Model) es la habilidad más crítica para cualquier QA Automation. Si el localizador falla, la prueba falla.

1. Introducción a los Localizadores

Selenium interactúa con el navegador buscando elementos HTML utilizando estrategias denominadas localizadores (**Locators**). En Java, esto se maneja mediante la clase **By** .

[↑ Volver al Índice General](#)

2. Estrategias Básicas de Localización

1. **By.id()** (La Mejor Opción): El ID de un elemento HTML debe ser único en toda la página según los estándares web. Es el método más rápido y seguro.

◦ Ejemplo: `<input id="correo">` -> `driver.findElement(By.id("correo"));`

2. **By.name()** (Segunda Mejor Opción): Muy común en campos de formularios. Generalmente son únicos, pero en grupos de "Radio Buttons" pueden repetirse.

◦ Ejemplo: `<input name="password">` -> `driver.findElement(By.name("password"));`

3. **By.className()** : Útil para encontrar listas de elementos similares, pero peligroso para elementos únicos porque las clases CSS cambian constantemente o se reutilizan.

◦ Ejemplo: `<button class="btn btn-primary">` -> `driver.findElement(By.className("btn-primary"));`

[↑ Volver al Índice General](#)

3. Selectores Avanzados: CSS Selectors (**By.cssSelector**)

Si no tienes ID ni Name, el CSS Selector es el rey de la velocidad. Se lee de izquierda a derecha.

- **Por ID:** Usando `#` . Ejemplo: `By.cssSelector("#correo")`
- **Por Clase:** Usando `.` . Ejemplo: `By.cssSelector(".btn-primary")`
- **Por Atributo Específico:** `[atributo='valor']` . Ejemplo: `By.cssSelector("input[type='submit']")`
- **Relaciones Hija Directa:** `div > p` (Encuentra el párrafo que está directamente dentro del div).
- **Subcadena (Contiene):** `[id*='dinamico']` (Encuentra un ID que contenga la palabra "dinamico", útil para IDs autogenerados como `dinamico_12345`).

[↑ Volver al Índice General](#)

4. Selectores Avanzados: XPath (**By.xpath**)

XPath es el lenguaje más poderoso, capaz de moverse hacia adelante, hacia atrás y navegar jerarquías complejas. Pero también es el más lento.

- **XPath Absoluto (NO USAR):** `/html/body/div[2]/form/input` . Si la interfaz cambia mínimamente, el XPath se rompe.
- **XPath Relativo (EXCELENTE):** `//input[@id='correo']` . Busca en todo el DOM sin importar dónde esté anidado.
- **Búsqueda por Texto (El superpoder de XPath):** `//button[text()='Ingresar']` . Localiza un botón por su texto visible.

- **Ejes (Axes):** Navegación relativa. Ej: `//td[text()='Juan']/following-sibling::td` (Encuentra la celda que está inmediatamente después de la celda que dice "Juan", útil para leer tablas web).

[↑ Volver al Índice General](#)

5. CSS Selector vs XPath: ¿Cuál elegir?

- **Velocidad:** CSS Selector es nativo del motor de los navegadores y es significativamente más rápido que XPath.
- **Legibilidad:** CSS suele ser más corto y limpio.
- **Poder:** Solo XPath puede localizar por texto o navegar "hacia arriba" (del hijo al padre).
- **Veredicto:** Usa `ID/Name` siempre que existan. Si no, intenta usar `CSS`. Si necesitas ubicar algo por su texto o navegar relaciones complejas, usa `XPath`.

[↑ Volver al Índice General](#)

6. Mejores Prácticas de QA (Framework FSelenium)

En nuestro framework, todos los localizadores deben estar declarados en la parte superior de las clases de tipo `Page Object` como variables privadas:

```
public class PaginaLogin extends BasePage {  
    private By txtUsuario = By.id("user");  
    private By btnIngresar = By.cssSelector(".btn-login");  
    // ...  
}
```

Esto garantiza que si el equipo de Frontend cambia el ID, solo debamos actualizar una sola línea en todo el proyecto.

[↑ Volver al Índice General](#)

En esta sección recopilamos los consejos más importantes para la supervivencia diaria en la automatización de pruebas con F Selenium.

1. El Flujo de Ejecución de Cucumber (BDD)

Cuando ejecutas una prueba BDD, la magia ocurre en tres capas:

1. **El Feature:** Escrito en Gherkin (`Given` , `When` , `Then`). Es el idioma de negocio.
2. **El Step Definition:** Código Java que enlaza el Gherkin con tus métodos técnicos mediante anotaciones como `@When("el usuario hace clic")` .
3. **El Page Object:** Donde reside realmente la interacción de Selenium (`driver.findElement(...)`).

Tip: ¡Nunca pongas un `driver.findElement` dentro de un Step Definition! Eso rompe la arquitectura limpia.

[↑ Volver al Índice General](#)

2. El Patrón Page Object Model (POM) con Herencia

Todo Page Object debe extender de `BasePage` . ¿Por qué? Porque `BasePage` ya tiene el `WebDriver` instanciado y todos los métodos genéricos (como `click()` , `escribir()`) envueltos en esperas explícitas automáticas.

- **Malo:** `driver.findElement(By.id("login")).click();`
- **Senior:** `click(botonLogin);` // Heredado de `BasePage`, incluye `WebDriverWait` internamente.

[↑ Volver al Índice General](#)

3. Manejo de Esperas: La Regla de Oro

Jamás uses `Thread.sleep(5000);` . Eso congela tu programa sin importar si el elemento cargó en el segundo 1. Multiplica eso por 100 pruebas y tendrás horas de tiempo muerto. Usa **Esperas Explícitas** (`WebDriverWait`):

```
WebDriverWait wait = new WebDriverWait(driver, Duration.ofSeconds(10));
wait.until(ExpectedConditions.visibilityOfElementLocated(By.id("dinamico")));
```

Esto esperará *hasta* 10 segundos, pero si carga al segundo 2, avanza instantáneamente.

[↑ Volver al Índice General](#)

4. Consejos de Depuración en IntelliJ

Cuando recibes un `NoSuchElementException` :

1. ¿El elemento está dentro de un `iframe` ? Si es así, debes usar `driver.switchTo().frame(...)` antes de buscarlo.
2. ¿Se abrió una nueva pestaña? Usa `driver.getWindowHandles()` para cambiar el foco.
3. Si la prueba va muy rápido, usa el modo **Debug** de IntelliJ poniendo un punto de interrupción (punto rojo) al lado de la línea conflictiva para frenar la ejecución y revisar la web en tiempo real.

[!\[\]\(082f818d99f166a3ba574d9284d73064_img.jpg\) Volver al Índice General](#)